Lecture 3 — Arrays and Dynamic Arrays

An array is a contiguous block of memory holding elements of the same type. The address of element i is base_address plus i times the element size, which is why random access is O(1): the arithmetic is independent of the array length.

Insertion into the middle of an array is O(n) because every element after the insertion point must be shifted right by one slot. Deletion from the middle has the same cost for the opposite reason.

A dynamic array hides the resizing problem from the caller. When the underlying array is full, a new array of double the capacity is allocated, all elements are copied over, and the old memory is freed.

The doubling strategy is what makes append amortized O(1). Although a single resize copies n elements, each element participates in only a logarithmic number of resizes over the lifetime of the array, so the average cost per append stays constant.

Trade-offs: arrays win on cache locality and random access but lose on arbitrary insertion and deletion. This is the fundamental tension we return to when we compare contiguous storage with linked storage next.

Lecture 4 — Linked Lists

A linked list stores elements in nodes that are not necessarily adjacent in memory. Each node holds its data plus a pointer to the next node; a doubly linked list also keeps a pointer to the previous node.

Because nodes are allocated separately, inserting at the head (or after a given node) is O(1) — you only relink a constant number of pointers, no shifting required. The same applies to deletion when you already hold the node to remove.

The price is that random access becomes O(n): finding element k requires walking the list node by node. Linked lists also lose cache locality, since nodes are scattered across memory rather than stored contiguously.

Space overhead is another cost: every node carries one or two extra pointers, so a linked list of small integers can use several times more memory than an array holding the same data.

Practical guidance: if your workload is dominated by appends and random reads, use a dynamic array. If it is dominated by insertions and deletions at known positions, a linked list can be the right tool.